

Programare Orientată Pe Obiecte

Curs 14: Testarea și depanarea programelor Structura Examen

Ianuarie 2026

Introducere

De ce testam software-ul?

Beneficii:

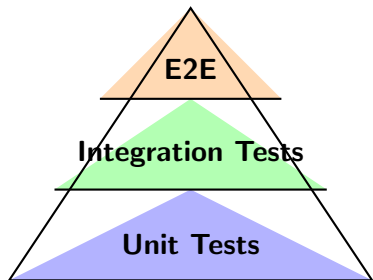
- Detectarea bug-urilor devreme
- Documentatie vie a codului
- Refactorizare cu incredere
- Reduce costurile pe termen lung
- Creste calitatea produsului

Costul bug-urilor:

- In dezvoltare: **1x**
- In testare: **10x**
- In productie: **100x**

*“Testarea arata prezenta bug-urilor,
nu absenta lor.” — Dijkstra*

Piramida Testarii



Lente, putine, costisitoare

Medii, moderate

Rapide, multe, ieftine

Tipuri de Testare

- **Unit Testing:** Testarea unei singure unitati (metoda/clasa) in izolare
- **Integration Testing:** Testarea interactiunii intre componente
- **End-to-End (E2E):** Testarea intregului flux al aplicatiei
- **Black-box Testing:** Testare fara cunoasterea implementarii
- **White-box Testing:** Testare cu acces la codul sursa
- **Regression Testing:** Verificarea ca modificarile nu au stricat functionalitati existente

JUnit 5

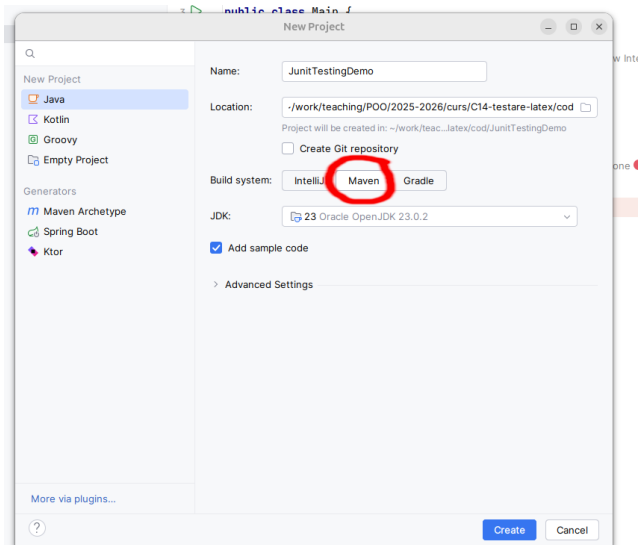
Ce este JUnit 5?

- Framework-ul standard pentru testare in Java
- Versiunea 5 = **JUnit Platform** + **JUnit Jupiter** + **JUnit Vintage**
- Suport nativ in IntelliJ IDEA
- Adnotari expresive si usor de folosit

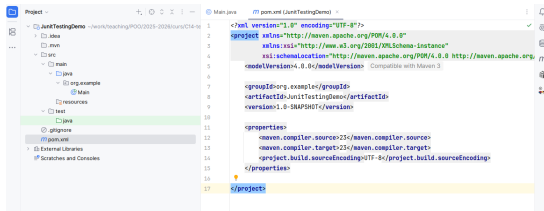
Dependenta Maven:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>
```

Creati un proiect nou - selectati Maven



Structura proiectului rezultat



Fisierul pom.xml

- Definește identitatea proiectului
- Gestionează dependențele — spune Maven ce biblioteci externe să descarce automat
- Configurează plugin-uri — pentru compilare, testare, rapoarte (JaCoCo), etc.
- Definește ciclul de build — ce se întâmplă când rulezi mvn compile, mvn test, mvn package

Creați o clasă nouă (Numere)

The screenshot shows an IDE interface with a project structure on the left and a code editor on the right.

Project Structure (Left Panel):

- Project: JUnitTestingDemo ~\work\teaching\POO
 - .idea
 - .mvn
 - src
 - main
 - java
 - org.example
 - Main
 - Numere** (selected)
 - resources
 - test
 - java
 - .gitignore
 - pom.xml
 - External Libraries
 - Scratches and Consoles

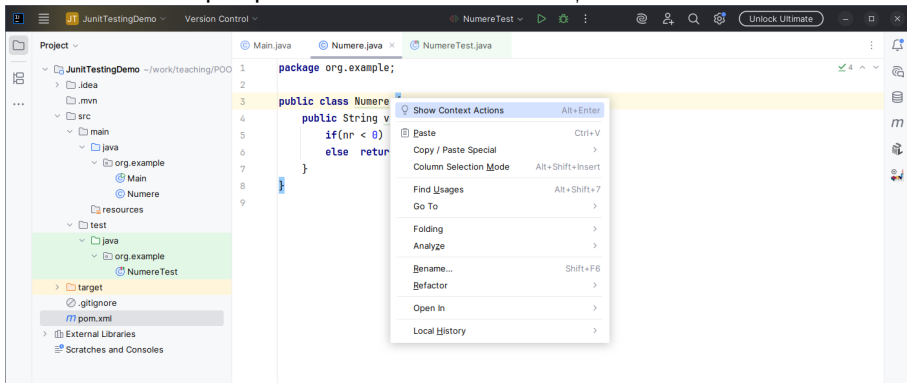
Code Editor (Right Panel):

The code editor shows the content of `Numere.java`:

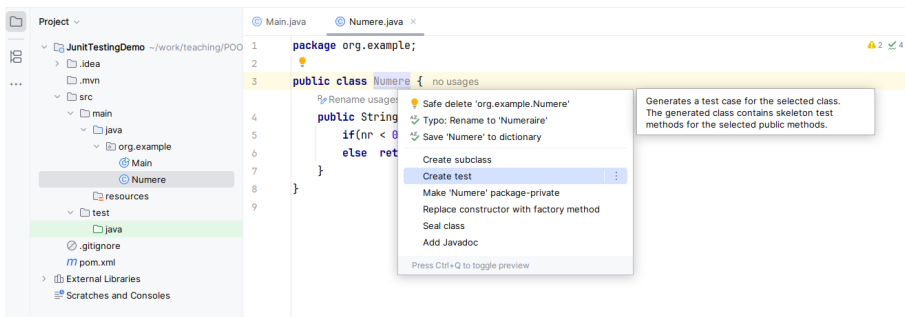
```
1 package org.example;  
2  
3 public class Numere { no usages  
4     public String verifica(int nr){  
5         if(nr < 0) return "negativ";  
6         else return "pozitiv";  
7     }  
8 }  
9
```

Context Actions:

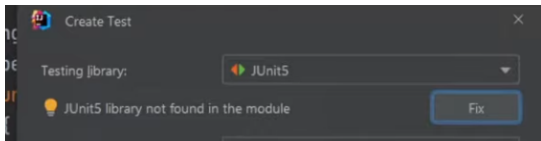
Click dreapta pe numele clasei si selectați "Context Actions":



Generate Test:



- Selectati "Create Test"
- Dacă apare mesajul "JUnit5 library not found in the module" apăsați Fix si apoi butonul OK.



Se generează clasa NumereTest:

Scrieți testele și rulați clasa NumereTest:

The screenshot shows an IDE window with the following components:

- Project Explorer (Left):** Displays the project structure. The `NumereTest` class is highlighted under `org.example` in the `test` directory.
- Code Editor (Center):** Shows the source code of `NumereTest.java`. The code includes imports for JUnit 5, a static import for `Assertions`, and two test methods: `getVerificaNegativTest()` and `getVerificaPozitivTest()`. Both methods create a `Numere` object and verify its `verifica` method returns the expected string ("negativ" or "pozitiv").
- Run Console (Bottom):** Shows the execution results. It indicates that 2 tests passed out of 2 total, taking 19ms. The command executed was `/home/raluca/.jdk/openjdk-23.0.2/bin/java ...`. The process finished with exit code 0.

```
1 package org.example;
2
3 import org.junit.jupiter.api.Assertions;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 class NumereTest {
9     @Test
10     void getVerificaNegativTest(){
11         Numere numere = new Numere();
12         String rezultat = numere.verifica(nr: -1);
13         Assertions.assertEquals( expected: "negativ", rezultat);
14     }
15
16     @Test
17     void getVerificaPozitivTest(){
18         Numere numere = new Numere();
19         String rezultat = numere.verifica(nr: 1);
20         Assertions.assertEquals( expected: "pozitiv", rezultat);
21     }
22 }
```

Run NumereTest x

✓ NumereTest (org.example) 19ms ✓ 2 tests passed 2 tests total, 19ms

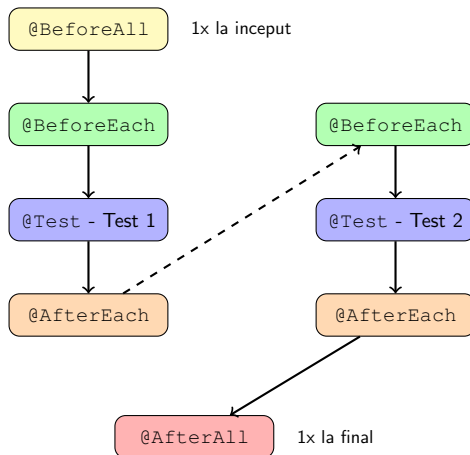
- ✓ getVerificaPozitivTest() 8 ms
- ✓ getVerificaNegativTest 11 ms

/home/raluca/.jdk/openjdk-23.0.2/bin/java ...

Process finished with exit code 0

Ce sunt Lifecycle Hooks?

Lifecycle Hooks = metode speciale care se executa automat in anumite momente ale ciclului de viata al testelor.



Cele 4 Lifecycle Hooks

Adnotare	Cand ruleaza	Utilizare
@BeforeAll	1x inainte de toate testele	Conexiuni DB, setup costisitor
@BeforeEach	Inainte de <i>fiecare</i> test	Instanta noua, stare curata
@AfterEach	Dupa <i>fiecare</i> test	Curatare resurse, reset
@AfterAll	1x dupa toate testele	Inchidere conexiuni, cleanup

Important:

- @BeforeAll si @AfterAll trebuie sa fie **static**
- @BeforeEach si @AfterEach **nu** sunt static
- Fiecare test primeste o instanta “fresh” a clasei de test

Exemplu: Clasa Numere

Clasa de testat:

```
package org.example;

public class Numere {
    public String verifica(int nr) {
        if (nr < 0) return "negativ";
        else return "pozitiv";
    }
}
```

Problema fara lifecycle hooks:

```
@Test
void testNegativ() {
    Numere numere = new Numere(); // Repetam in fiecare test!
    assertEquals("negativ", numere.verifica(-1));
}

@Test
void testPozitiv() {
    Numere numere = new Numere(); // Din nou...
    assertEquals("pozitiv", numere.verifica(1));
}
```

Exemplu: NumereTest cu Lifecycle Hooks

```
class NumereTest {
    private Numere numere; // Variabila la nivel de clasa

    @BeforeAll
    static void initAll() {
        System.out.println("== Incepem testarea clasei Numere ==");
    }

    @BeforeEach
    void setUp() {
        numere = new Numere(); // Instanta noua pentru fiecare test
        System.out.println("Am creat o instanta noua de Numere");
    }

    @Test
    void testNegativ() {
        assertEquals("negativ", numere.verifica(-1));
    }

    @Test
    void testPozitiv() {
        assertEquals("pozitiv", numere.verifica(1));
    }
}
```

Exemplu: NumereTest cu Lifecycle Hooks (cont.)

```

@Test
void testZero() {
    assertEquals("pozitiv", numere.verifica(0));
}

@AfterEach
void tearDown() {
    System.out.println("Test finalizat, curatam resursele");
    numere = null; // Eliberam referinta
}

@AfterAll
static void tearDownAll() {
    System.out.println("== Am terminat testarea clasei Numere ==");
}
}
    
```

Output la rulare

Cand rulam testele, output-ul arata astfel:

```
== Incepem testarea clasei Numere ==  
Am creat o instanta noua de Numere  
Test finalizat, curatam resursele  
Am creat o instanta noua de Numere  
Test finalizat, curatam resursele  
Am creat o instanta noua de Numere  
Test finalizat, curatam resursele  
== Am terminat testarea clasei Numere ==
```

Observati:

- @BeforeAll si @AfterAll — cate o singura data
- @BeforeEach si @AfterEach — de 3 ori (cate o data pentru fiecare test)

De ce sunt utile Lifecycle Hooks?

1. Elimina duplicarea codului

- Nu mai repetam `new Numere()` in fiecare test
- Codul e mai curat si mai usor de intretinut

2. Garanteaza independenta testelor

- Fiecare test primeste o instanta “fresh”
- Un test nu poate afecta alt test
- Ordinea de rulare nu conteaza

3. Gestioneaza resursele corect

- Setup costisitor (conexiuni DB) — o singura data cu `@BeforeAll`
- Curatare dupa fiecare test — `@AfterEach`
- Eliberare resurse la final — `@AfterAll`

4. Urmeaza principiul DRY (Don't Repeat Yourself)

Testele parametrizate

Problema: Vrem sa testam aceeași logica cu valori diferite.

Fara teste parametrizate — cod duplicat:

```
@Test void testNegativ1() {assertEquals("negativ", numere.verifica(-1))
    ;}
@Test void testNegativ2() {assertEquals("negativ", numere.verifica(-5))
    ;}
@Test void testNegativ3() {assertEquals("negativ", numere.verifica(-100))
    );}
@Test void testPozitiv1() {assertEquals("pozitiv", numere.verifica(1));}
@Test void testPozitiv2() {assertEquals("pozitiv", numere.verifica(5));}
// ... si tot asa pentru fiecare valoare
```

Solutia: `@ParameterizedTest` — un singur test, mai multe valori!

Beneficii:

- Eliminam duplicarea codului
- Adaugam usor cazuri noi de test
- Codul e mai lizibil si mai usor de intretinut

Surse de Date pentru Teste Parametrizate

Adnotare	Descriere
@ValueSource	Lista simpla de valori (int, String, etc.)
@CsvSource	Perechi/tuple de valori separate prin virgula
@CsvFileSource	Date din fisier CSV extern
@MethodSource	Date generate de o metoda Java
@EnumSource	Toate valorile unui enum
@NullSource	Valoarea null
@EmptySource	Valoare goala (String gol, lista goala)

Dependentia necesara (junit-jupiter-params):

```
groupId:  org.junit.jupiter
artifactId:  junit-jupiter-params
version:  5.10.2
```

Exemplu: NumereTest cu @ValueSource

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

class NumereTest {
    private Numere numere = new Numere();

    @ParameterizedTest
    @ValueSource(ints = {-1, -5, -10, -100, -999})
    @DisplayName("Testam mai multe numere negative")
    void testNumereNegative(int numar) {
        assertEquals("negativ", numere.verifica(numar));
    }

    @ParameterizedTest
    @ValueSource(ints = {1, 5, 10, 100, 999})
    @DisplayName("Testam mai multe numere pozitive")
    void testNumerePozitive(int numar) {
        assertEquals("pozitiv", numere.verifica(numar));
    }
}
```

Rezultat: 10 teste rulate (5 negative + 5 pozitive) din doar 2 metode!

Exemplu: NumereTest cu @CsvSource

@CsvSource permite sa specificam **input si output asteptat**:

```
@ParameterizedTest
@CsvSource({
    "-1, negativ",
    "-100, negativ",
    "0, pozitiv",
    "1, pozitiv",
    "100, pozitiv"
})
@DisplayName("Testam verifica() cu diverse valori")
void testVerificaCuCsv(int numar, String rezultatAsteptat) {
    assertEquals(rezultatAsteptat, numere.verifica(numar));
}
```

Cum functioneaza:

- Fiecare rand din @CsvSource = un test separat
- Valorile sunt separate prin virgula
- Prima valoare → parametrul `numar`
- A doua valoare → parametrul `rezultatAsteptat`

Exemplu: NumereTest cu @MethodSource

Pentru cazuri complexe, putem genera datele dintr-o metoda:

```
import java.util.stream.Stream;
import org.junit.jupiter.params.provider.Arguments;
import org.junit.jupiter.params.provider.MethodSource;

@ParameterizedTest
@MethodSource("dateDeTest")
@DisplayName("Testam cu date din metoda")
void testVerificaCuMethodSource(int numar, String rezultatAsteptat) {
    assertEquals(rezultatAsteptat, numere.verifica(numar));
}

static Stream<Arguments> dateDeTest() {
    return Stream.of(
        Arguments.of(-50, "negativ"),
        Arguments.of(-1, "negativ"),
        Arguments.of(0, "pozitiv"),
        Arguments.of(1, "pozitiv"),
        Arguments.of(Integer.MAX_VALUE, "pozitiv")
    );
}
```

Output in IntelliJ

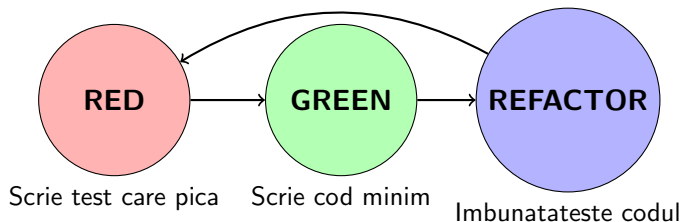
Cand rulam testele parametrizate, IntelliJ afiseaza fiecare valoare:

```
NumereTest
  testNumereNegative(int)
    [1] -1      PASSED
    [2] -5      PASSED
    [3] -10     PASSED
    [4] -100    PASSED
    [5] -999    PASSED
  testNumerePozitive(int)
    [1] 1       PASSED
    [2] 5       PASSED
    [3] 10      PASSED
    [4] 100     PASSED
    [5] 999     PASSED
  testVerificaCuCsv(int, String)
    [1] -1, negativ PASSED
    [2] -100, negativ PASSED
    [3] 0, pozitiv PASSED
    [4] 1, pozitiv PASSED
    [5] 100, pozitiv PASSED
```


TDD

Ce este TDD?

Test-Driven Development = Scriem testele *inainte* de cod!



Beneficiile TDD

Avantaje:

- Design mai bun al codului
- Cod testabil by default
- Documentatie automata
- Mai putine bug-uri
- Refactorizare sigura

Reguli TDD:

- 1 Nu scrie cod fara un test care pica
- 2 Scrie doar atat test cat sa pice
- 3 Scrie doar atat cod cat sa treaca testul

31 / 55

TDD in Practica - Pas 2: GREEN

```
// Pasul 2: Scriem MINIM cod ca testul sa treaca
public class ContBancar {
    private double sold = 0;

    public double getSold() {
        return sold;
    }
}

// Rulam testul: VERDE! Trecem la urmatorul test.

// Test nou pentru depunere
@Test
void depunereCresteSoldul() {
    ContBancar cont = new ContBancar();
    cont.depune(100);
    assertEquals(100, cont.getSold());
}
```

TDD - Testarea Edge Cases

```

@Test
@DisplayName("Nu se poate retrage mai mult decat soldul")
void retragereInsuficienta() {
    ContBancar cont = new ContBancar();
    cont.depune(50);

    assertThrows(
        SoldInsuficientException.class,
        () -> cont.retrage(100)
    );
    assertEquals(50, cont.getSold());
}

@Test
void depunereSumaNegativaArunca() {
    ContBancar cont = new ContBancar();
    assertThrows(IllegalArgumentException.class,
        () -> cont.depune(-50));
}

```

Mockito

Ce este Mocking?

Problema: Cum testam o clasa care depinde de alte clase/servicii?

- **Mock** = Obiect fals care simuleaza comportamentul unui obiect real
- Permite testarea in **izolare**
- Evitam dependentele externe (baze de date, API-uri, etc.)
- **Mockito** = Cel mai popular framework de mocking pentru Java

Dependenta Maven:

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-jupiter</artifactId>
  <version>5.11.0</version>
  <scope>test</scope>
</dependency>
```

Code Coverage

Ce este code coverage?

Code Coverage = Procentul de cod executat de teste

Tipuri de coverage:

- **Line Coverage:** Ce procent din linii sunt executate?
- **Branch Coverage:** Sunt testate toate ramurile if/else?
- **Method Coverage:** Ce procent din metode sunt apelate?
- **Class Coverage:** Ce procent din clase sunt testate?

Instrumente populare:

- **JaCoCo** - integrat in Maven/Gradle
- **IntelliJ IDEA** - built-in coverage

Coverage in IntelliJ IDEA

Cum se rulează testele cu coverage:

- 1 Click dreapta pe test/clasa de test
- 2 Selectati **“Run with Coverage”**
- 3 Sau: **Ctrl+Shift+F10** apoi selectati “with Coverage”

Interpretarea rezultatelor:

- **Verde**: Linie executata de teste
- **Rosu**: Linie neexecutata
- **Galben**: Branch partial acoperit

Tinte recomandate: 80% pentru business logic, 100% pentru cod critic

Logging

De ce avem nevoie de Logging?

- Diagnosticare: Intelegerea comportamentului in productie
- Debugging: Identificarea cauzei problemelor
- Audit: Inregistrarea actiunilor importante
- Monitorizare: Alertare la erori

Evolutia logging-ului in Java:

- `System.out.println()` - **NU in productie!**
- `java.util.logging` - Built-in, dar verbose
- SLF4J + Logback - Standard modern
- Log4j 2 - Performant, configurabil

Niveluri de Logging

Nivel	Cand folosim	Exemplu
ERROR	Erori grave	Conexiune DB pierduta
WARN	Situatii neasteptate	Retry dupa timeout
INFO	Evenimente importante	User autentificat
DEBUG	Informatii debugging	Valori variabile
TRACE	Detalii foarte fine	Intrare/iesire metode

In productie: INFO sau WARN — In development: DEBUG sau TRACE

SLF4J + Logback

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>2.0.12</version>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.5.3</version>
</dependency>
```

Logging in Practica

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class ContBancar {
    private static final Logger logger =
        LoggerFactory.getLogger(ContBancar.class);

    public void depune(double suma) {
        logger.info("Depunere initiata: {} RON", suma);

        if (suma <= 0) {
            logger.warn("Suma invalida: {}", suma);
            throw new IllegalArgumentException("Suma > 0");
        }

        this.sold += suma;
        logger.debug("Sold actualizat: {} RON", this.sold);
    }
}
```

Logging cu Lombok

```
import lombok.extern.slf4j.Slf4j;

@Slf4j // Genereaza automat logger-ul!

public class ContBancar {

    public void retrage(double suma) {
        log.info("Retragere initiata: {} RON", suma);

        if (suma > sold) {
            log.error("Sold insuficient! Cerut: {}, " +
                    "Disponibil: {}", suma, sold);
            throw new SoldInsuficientException("Insuficient");
        }

        sold -= suma;
        log.info("Retragere OK. Sold: {} RON", sold);
    }
}
```

Debugging

De ce avem nevoie de Debugger?

- Logging-ul nu e suficient pentru probleme complexe
- Debugger-ul permite:
 - Oprirea executiei la un anumit punct
 - Inspectarea valorilor variabilelor
 - Executie pas cu pas
 - Modificarea valorilor in runtime

Concepte cheie:

- **Breakpoint:** Punct de oprire in cod
- **Step Over (F8):** Executa linia curenta
- **Step Into (F7):** Intra in metoda apelata
- **Step Out (Shift+F8):** Iesi din metoda curenta
- **Resume (F9):** Continua pana la urmatorul breakpoint

Breakpoints in IntelliJ

Tipuri de Breakpoints:

- **Line Breakpoint:** Click in marginea stanga
- **Conditional Breakpoint:** Click dreapta pe breakpoint (ex: suma > 1000)
- **Exception Breakpoint:** Se opreste la exceptii
- **Method Breakpoint:** La intrarea/iesirea din metoda

Shortcut-uri importante:

- **Shift+F9:** Start Debug
- **Ctrl+F8:** Toggle Breakpoint
- **Ctrl+Shift+F8:** Vezi toate breakpoints
- **Alt+F8:** Evaluate Expression

Panoul Debug in IntelliJ

Sectiuni importante:

- **Frames:** Stack-ul de apeluri (call stack)
- **Variables:** Variabilele din scope-ul curent
- **Watches:** Expresii personalizate de urmarit
- **Console:** Output si input

Functii avansate:

- **Evaluate Expression (Alt+F8):** Evalueaza orice expresie Java
- **Set Value:** Modifica valoarea unei variabile
- **Force Return:** Forteza returnarea dintr-o metoda
- **Drop Frame:** "Deruleaza" inapoi la frame-ul anterior

Best Practices

Structura unui Proiect cu Teste

Structura Maven standard:

- src/main/java/ - Cod sursa
- src/test/java/ - Teste
- src/test/resources/ - Resurse test

Conventii de numire:

- Calculator.java
- CalculatorTest.java

Exemplu structura: project/

```
src/
  main/java/
    com/example/
      Calculator.java
  test/java/
    com/example/
      CalculatorTest.java
pom.xml
```

Principii FIRST pentru Teste

- **F**ast - Testele trebuie sa ruleze rapid
- **I**ndependent - Testele nu depind unul de altul
- **R**epeatable - Acelasi rezultat de fiecare data
- **S**elf-validating - Pass/Fail clar, fara interpretare manuala
- **T**imely - Scrise la momentul potrivit (ideal: TDD)

Resurse Suplimentare

Documentatie oficiala:

- JUnit 5: <https://junit.org/junit5/docs/current/user-guide/>
- Mockito: <https://site.mockito.org/>
- SLF4J: <https://www.slf4j.org/manual.html>

Carti recomandate:

- “Test Driven Development” - Kent Beck
- “Clean Code” - Robert C. Martin

Practica: <https://exercism.org/tracks/java>

Examen

Structura examenului

- Problema 1: ce se afișează la rularea unui cod dat
- Problema 2: întrebări de tipul True / False
- Problemele 3, 4, 5 : teorie (definire si explicații concepte studiate la curs)
- Problema 6: să se scrie codul pentru o aplicație JAVA
 - Exemplu: Scrieți codul JAVA complet pentru a simula jucarea unor meciuri în sporturile de echipă (VOLEI, FOTBAL, BASCHET, POLO) conform specificațiilor date.
 - Identificați și definiți clasele (cu metode, attribute) relevante pentru rezolvarea cerințelor.
 - Utilizați în cod următoarele: mostenire, clasă abstractă, polimorfism, interfață, variabilă statică, apel al constructorului din clasa părinte
 - Codul va cuprinde și o clasă cu metoda main() în care se vor crea mai mulți jucători, se vor crea echipe și meciuri.
 - Desenați diagrama UML de clase pentru aplicație.

Feedback Curs si Laborator

Feedback

Succes în sesiune !